

1. Problem Statement

1.1 Objective

Design and implement a Retrieval-Augmented Generation (RAG) workflow for AeroVelo Logistics Global. Given an operational question, the assistant should retrieve relevant passages from the supplied documents and generate an answer grounded in that context. Learners load and analyse the documents, create overlapping chunks, build a vector database, connect a language model and evaluate the responses.

1.2 Business Value

Operations teams search across supplier policies, incident procedures, financial authorisation rules and compliance documents. A document-grounded assistant can reduce manual searching and help staff obtain consistent information from the available sources. Displaying retrieved filenames supports traceability. A clear response when information is unavailable reduces unsupported claims. This assignment provides information only and does not execute business transactions.

2. Dataset Overview

2.1 Context

The dataset describes the fictional organisation AeroVelo Logistics Global. It contains 10 files: nine Markdown documents and one PDF, covering seven business functions. Files are organised into eight folders. The collection supports a compact exercise in document loading, semantic retrieval and grounded question answering.

2.2 Content

File or folder	Contents and use
compliance/	One Markdown document covering regulatory breach reporting.
finance/	One Markdown document covering financial authorisation and procurement.
governance/	One Markdown document containing the regional governance matrix.
hr/	Two Markdown documents covering US and EMEA remote working policies.
legal/	One Markdown document covering supplier personal data processing.
operations/	One Markdown document covering incident management.
pdfs/	One PDF version of the regional governance matrix.
supply_chain/	Two Markdown documents covering supplier qualification and the Bill of Materials.

The regional governance matrix is supplied in both Markdown and PDF formats. Inspect this content overlap when comparing documents and interpreting retrieval results. The five evaluation questions and reference answers are defined within the notebook, rather than supplied in a separate benchmark file.

3. Instructions

1. This is a group assignment. Complete the assignment with Python in one Jupyter notebook using the supplied dataset.
2. Extract the dataset ZIP into a dataset folder beside the notebook. Retain its original subfolder structure.

3. Prepare Ollama and the llama3.2:3b model before starting. The package installation commands are included as commented lines in the final solution notebook; uncomment and run them when required. Setup carries no separate marks.
4. Gemini is an optional alternative. Install langchain-google-genai only if using it, and keep any API key out of the submitted notebook.
5. Load Markdown and PDF content with filename, category, file type and source path metadata. Clean whitespace without changing the meaning of the documents.
6. Calculate document-length statistics, analyse word frequencies and compare documents using TF-IDF vectors and cosine similarity.
7. Create overlapping chunks using a chunk size of 800 characters and an overlap of 120 characters. Preserve metadata on each chunk.
8. Generate embeddings with sentence-transformers/all-MiniLM-L6-v2 and populate a persistent Chroma collection named logistics_rag.
9. Retrieve four chunks per question. Connect the retriever, context formatter, grounded prompt and local language model, using temperature 0.
10. Evaluate five paired questions and reference answers. Calculate answer similarity and retrieval similarity, summarise their averages and test one unsupported question.
11. Display retrieved source filenames alongside generated answers. Record model/runtime assumptions and explain observed limitations.

4. Submission Guidelines

- Include the assignment title and your group name near the start of the notebook.
- Submit complete Jupyter notebook in .pynb format. Suggest filename: Logisitic_Operation_RAG <your_group_name>.ipynb
- Include runnable code, visible outputs, document analysis, chunking and retrieval examples, evaluation results and conclusions.
- Keep the dataset folder structure available to the notebook. Do not embed the full dataset or private credentials in the submission.
- State the embedding model, language model and setup assumptions.
- Check reproducibility with a fresh assignment vector collection before indexing. Re-running an append-only ingestion cell can create duplicate vectors.
- Report failures and limitations honestly. Keep evaluation questions paired with their reference answers and do not alter answers merely to improve scores.

5. Results Expected from Learners

A completed submission should provide a working notebook that demonstrates these outcomes:

- Load all 10 documents, preserve their source metadata and clean unnecessary whitespace.
- Report minimum, maximum and average document lengths, the 20 most and least frequent non-stop words, and a TF-IDF document similarity matrix.
- Create overlapping chunks, generate embeddings and store them in a Chroma vector database.
- Build a working RAG chain that retrieves four chunks and generates an answer using the retrieved context.
- Answer operational questions about supplier performance, incidents, regulatory reporting, supplier data processing and financial authorisation.
- Return generated answers and display the filenames of the retrieved source documents. Acknowledge when the context does not contain an answer.

- Evaluate five supported questions and one unsupported question. Summarise the similarity scores and explain representative limitations.

Exact response wording can vary between models. Grading focuses on implementation, relevant retrieval, supported answers, traceability and interpretation rather than exact phrase matching.

5.1 Assessment Checkpoints and Marks

Checkpoint	Expected work	Marks
1.2.1 Load Logistics Documents	Load all Markdown (.md) and PDF (.pdf) documents from the dataset folder. Store useful metadata such as the file name, business category, file type, and source path for later retrieval and evaluation.	7
1.2.2 Clean and Normalise Document Text	Clean the loaded document text by removing unnecessary whitespace and formatting inconsistencies while preserving the original document meaning and metadata.	5
1.3.1 Analyse Document Lengths	Calculate the average, maximum and minimum document length.	4
1.3.2 Analyse Word Frequencies	Analyse the frequency of words in the logistics documents and identify the most and least frequently occurring words.	6
1.3.3 Compare Document Similarity	Analyse the similarity between different logistics documents using TF-IDF vectors and cosine similarity.	6
1.4.1 Create Overlapping Document Chunks	Perform appropriate steps to split the logistics documents into smaller overlapping chunks while preserving the document metadata.	12
2.1.1 Initialise the Embedding Model	Initialise an embedding function to generate vector representations of the document chunks.	5
2.1.2 Create and Populate the Vector Database	Load the document chunk embeddings into a Chroma vector database for semantic retrieval.	11
2.2.1 Build the RAG Pipeline	Form the complete RAG pipeline by connecting the vector database retriever, prompt and language model.	12
2.2.2 Generate Answers and Display Sources	Create a function to generate answers for logistics-related questions.	7
3.1.1 Prepare Evaluation Questions and Reference Answers	Create a small evaluation set containing logistics questions and their expected ground-truth answers.	4
3.1.2 Evaluate Answer and Retrieval Similarity	Create a function to evaluate the generated answers.	10
3.1.3 Interpret Results and Test Unsupported Questions	Draw inferences from the evaluation results and test how the RAG system handles a question that is not supported by the provided documents.	6
4.1.1 Summarise Findings and Limitations	Conclude with the results obtained from the data analysis, document chunking, vector retrieval, RAG pipeline and evaluation.	5
Total		100

6. Evaluation Rubrics

The criteria below describe how markers assess the work. Criterion weights align with the notebook sections and checkpoint marks. Award credit for correct implementation and evidence in the outputs; fluent wording alone does not earn marks.

Criteria	Evaluation Parameters
1. Data Loading, Preparation and Analysis (40 marks)	1. Loads Markdown and PDF content with metadata. 2. Cleans whitespace while preserving meaning. 3. Calculates length and word-frequency statistics. 4. Compares documents with TF-IDF cosine similarity. 5. Creates overlapping chunks with source metadata.
2. Vector Database and RAG Chain Creation (35 marks)	1. Initialises the specified embedding model. 2. Stores chunks in Chroma. 3. Connects the retriever, context formatter, prompt and LLM. 4. Retrieves four chunks and generates context-based answers. 5. Displays retrieved source filenames.
3. RAG Evaluation (20 marks)	1. Prepares five paired questions and reference answers. 2. Calculates answer and retrieval similarity. 3. Reports per-question results and average scores. 4. Tests an unsupported question and interprets limitations.
4. Conclusion (5 marks)	1. Summarises findings supported by observed outputs. 2. Explains limitations of retrieval and evaluation. 3. Identifies a practical improvement.

Total assignment marks: 100. Answer similarity compares the generated answer with its reference answer using embedding cosine similarity. Retrieval similarity is the highest similarity between the reference answer and the retrieved chunks. These scores are diagnostic measures, not accuracy percentages or proof of factual grounding.

6.1 Performance Guidance

Performance level	General standard
Strong	The complete workflow is reproducible; retrieval and responses are relevant; evaluation and limitations are clearly interpreted.
Competent	The core pipeline works; most responses are supported; minor retrieval or evaluation details need improvement.
Developing	Some loading, retrieval or generation works, but the pipeline or evaluation is incomplete or inconsistent.
Insufficient	The notebook cannot demonstrate a working document-grounded RAG pipeline and meaningful evaluation.

7. Suggested Two to Two and a Half Hour Schedule

Time	Suggested focus
0–20 min	Load, inspect and clean the documents.
20–40 min	Analyse document lengths, word frequencies and document similarity.
40–60 min	Create chunks, generate embeddings and populate Chroma.
60–90 min	Build the RAG chain and test answers with source filenames.
90–115 min	Evaluate five supported questions and test one unsupported question.
115–135 min	Interpret results, write conclusions and check reproducibility.
135–150 min	Allow for troubleshooting and local inference delays.

8. Dataset and Runtime Notes

Install Ollama separately and download the local model with `ollama pull llama3.2:3b`. Prepare the embedding model and NLTK English stop words before the timed exercise. Inference speed and generated wording depend on the environment. The notebook answers questions from documents and does not connect to live logistics systems.